

实验三 栈和队列实验

栈基础练习一

【问题描述】

题目要求根据给出的顺序栈结构，完成初始化、显示栈顶元素、入栈、出栈、判空和判满等基本操作。本实验在 `Stacks/ex_1` 中实现定长顺序栈，最大容量设置为 1024，主程序演示了顺序入栈、读取栈顶和弹栈过程。

【基本思路】

存储结构直接按题目要求设计，使用长度为 1024 的整型数组保存栈内元素，再用 `top` 表示当前栈顶上方的位置。空栈时 `top=0`，入栈时先把元素写入 `data[top]`，随后 `top` 加 1；出栈时先把 `top` 减 1，再返回原栈顶元素。

`SeqStack::new()` -> `top <- 0`；`push(x)` 时若栈未滿则 `data[top] <- x`，`top <- top + 1`；`pop()` 时若栈非空则 `top <- top - 1`，再返回 `data[top]`。

函数调用关系可表示为：`main()` -> `SeqStack::new()` -> `push()/top()/pop()/output()`。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分采用 Rust 语言开发，核心顺序栈结构和操作函数位于 `Stacks/ex_1/src/lib.rs`，演示程序位于 `src/main.rs`，测试代码位于 `tests/test_fn.rs`。
测试数据重点覆盖空栈、正常入栈出栈、压满 1024 个元素后的判满行为，以及再次弹空后的边界情况。

1.2 测试结果

运行 `cargo test` 后，本题 2 个单元测试全部通过；运行 `cargo run` 后，程序输出顺序栈为空、入栈后为 `[10, 20, 30]`、栈顶为 30，弹出后剩余 `[10, 20]`，与预期一致。

编号	测试数据	预期结果 / 实际分析
1	依次入栈 10、20、30	输出 <code>[10, 20, 30]</code> ，栈顶元素为 30。
2	弹出 1 次后再查看栈内容	返回 30，剩余元素为 <code>[10, 20]</code> 。
3	连续压入 1024 个元素后再入栈	判定为满，后续 <code>push</code> 返回 <code>false</code> 。

2. 结果及方案分析

2.1 本实验的重点及难点

top 指针表示“下一个可用位置”，不是当前元素下标。若把这两层含义写混，入栈和出栈就容易出现越界。

2.2 本实验设计的优点及可以改进的方向

结构简单、判断直接，和教材结构几乎一一对应。可以改进的方向是把错误原因细分出来，例如分别区分“栈空”和“栈满”。

还有哪些实现方法？

用 `Vec<i32>` 模拟顺序栈。

【问题描述】

题目要求根据链结点结构构造一个链栈，并完成与表 10-8 相同的基本操作。本实验在 Stacks/ex_2 中用单链表头插法实现链栈，栈顶总是放在链表头部，因此入栈和出栈都可以在 O(1) 时间内完成。

【基本思路】

链栈使用 LinkNode 表示结点，结点中保存整数 info 和后继指针 next。每次 push 时新建结点并让它指向原来的头结点，再把 head 更新到新结点；每次 pop 时取下 head 并把 head 改到原来第二个结点。

由于链式存储不需要预先分配连续空间，所以本实现的 is_full() 固定返回 false。严格地说只有在内存耗尽时它才可能失败，这也是“链栈是否要判栈满”的答案。这里保留该接口，主要是为了和顺序栈操作保持一致。

push(x) 时 `node.next <- head, head <- node`；pop() 时取出 `head.info`，再令 `head <- head.next`。

函数调用关系可表示为：`main() -> LinkStack::new() -> push()/top()/pop()/output()/is_empty()`。

【实验结果及分析】

1. 实验结果：

1.1 实验过程描述

核心实现位于 Stacks/ex_2/src/lib.rs，主程序演示了连续压栈 5、15、25 后查看栈顶，再弹出一个元素的过程。

测试数据覆盖了空链栈、正常压栈、连续弹空，以及大量压栈后 is_full() 仍返回 false 的情况。

1.2 测试结果

运行 cargo test 后，本题 2 个单元测试全部通过；运行 cargo run 后，链栈内容从栈顶到栈底依次显示为 [25, 15, 5]，弹出 25 后剩余 [15, 5]。

编号	测试数据	预期结果 / 实际分析
1	压栈 5、15、25	输出 [25, 15, 5]，说明头插法构造正确。
2	弹出 1 次	返回 25，剩余 [15, 5]。
3	连续压入 200 个元素	链栈仍可正常工作，is_full() 返回 false。

2. 结果及方案分析

2.1 本实验的重点及难点

理解链栈要把链表头部当成栈顶，这样操作最方便。如果把尾部当栈顶，就要在单链表上频繁遍历。

2.2 本实验设计的优点及可以改进的方向

入栈和出栈都不需要搬移元素。可以改进的地方是目前 `output()` 每次都要从头遍历，如果数据量很大，展示时会稍慢一些。

【问题描述】

题目要求将单链表中 n 个结点逆置。本实验在 `Stacks/ex_3` 中先顺序扫描链表，把结点数据逐个压入辅助栈，再按照出栈次序重新构造新链表，从而得到逆序结果。

【基本思路】

采用借助栈逆置的方案，而不是直接就地反转指针。程序先遍历原链表，把每个 `data` 依次压入 `Vec` 形成后进先出的辅助栈，随后按栈中元素次序新建链表。

遍历原链表，把 `data` 依次压栈；然后循环弹栈，每弹出一个值就头插到新链表中。

函数调用关系可表示为：`main()` -> `from_slice()` -> `reverse_list()` -> `to_vec()` -> 输出逆置前后结果。

【实验结果及分析】

1. 实验结果：

1.1 实验过程描述

本题代码位于 `Stacks/ex_3/src/lib.rs`，辅助函数 `from_slice()` 和 `to_vec()` 仅用于快速构造测试链表、观察逆置结果。

测试数据覆盖了空链表、单结点链表和多结点链表，其中 `[1,2,3,4,5]` 逆置后应得到 `[5,4,3,2,1]`。

1.2 测试结果

运行 `cargo test` 后，本题 2 个单元测试全部通过；运行 `cargo run` 后，屏幕先输出逆置前 `[1, 2, 3, 4, 5]`，再输出逆置后 `[5, 4, 3, 2, 1]`。

编号	测试数据	预期结果 / 实际分析
1	输入链表 <code>[1,2,3,4,5]</code>	逆置后得到 <code>[5,4,3,2,1]</code> 。
2	输入空链表	输出仍为空链表，不会报错。
3	输入单结点 <code>[7]</code>	逆置后仍为 <code>[7]</code> ，边界情况正确。

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点不在链表构造本身，而在于如何利用“先进后出”把原序列翻转过来。因为链表中的 `next` 指针需要被安全移动，所以用 `Rust` 写时要特别注意 `take()` 的用法。

2.2 本实验设计的优点及可以改进的方向

这种设计的优点是好理解，也便于写单元测试。可以改进的方向是采用三指针法原地反转链表，那样空间复杂度可以降为 $O(1)$ 。

还有哪些实现方法？

若不限定必须体现栈结构，也可以直接迭代修改 `next` 指针完成逆置。

栈进阶练习二

【问题描述】

题目要求检查给定字符串是否为回文，忽略空白和其它符号，字母不区分大小写，并分别使用顺序栈和链栈实现。本实验在 `Stacks/ex_4` 中先把字符串规范化，只保留字母与数字并统一转为小写，再分别用两种栈逐一比较正向字符和反向字符。

【基本思路】

处理时先对原字符串做预处理。`normalize_text()` 会过滤掉空格、标点等非字母数字字符，再把剩余字符转成小写。随后顺序栈版本和链栈版本都把这些字符全部压栈，再依次和原字符序列比较。

如果所有位置都匹配，则为回文；只要出现一次不匹配，就立刻返回 `false`。这一部分和教材的思想一致，只是为了方便观察，又提供了两个独立函数 `is_palindrome_seq()` 和 `is_palindrome_link()`。题目给出的“Madam I am Adam”样例按当前规则实际不是严格回文，这里仍按程序真实结果记录。

`text` 先规范化为字符数组 `data`；`for ch in data: push(ch)`；再 `for ch in data: 若 pop() != ch 则不是回文`。

函数调用关系可表示为：`main() -> is_palindrome_seq()/is_palindrome_link() -> normalize_text() -> 输出判断结果`。

【实验结果及分析】

1. 实验结果：

1.1 实验过程描述

代码位于 `Stacks/ex_4/src/lib.rs`，主程序用题面样例和一个非回文字符串进行演示。为了比较两种存储结构，顺序栈和链栈版本保持了相同的判断流程。

测试数据覆盖了 `was it a cat I saw`、`Level`、`abcdef`、`12321` 等情况，同时检验了忽略大小写和符号后的结果是否正确。

1.2 测试结果

运行 `cargo test` 后，本题 2 个单元测试全部通过；运行 `cargo run` 后，程序输出 `was it a cat I saw` 和 `Level` 为 `true`，`abcdef` 为 `false`，而 `Madam I am Adam` 输出为 `false`。

编号	测试数据	预期结果 / 实际分析
1	"was it a cat I saw"	忽略空格后为回文，两种栈实现都返回 <code>true</code> 。
2	"Level"、"abcdef"	前者返回 <code>true</code> ，后者返回 <code>false</code> ，区分效果正确。
3	"Madam I am Adam"	程序返回 <code>false</code> ，说明按当前规范化

		规则它并非严格回文。
--	--	------------

2. 结果及方案分析

2.1 本实验的重点及难点

本题重点是先把“应该比较哪些字符”处理正确，再利用栈完成逆序比较。若预处理做错，后面再正确的栈操作也没有意义。

2.2 本实验设计的优点及可以改进的方向

设计优点是顺序栈和链栈两套实现可以互相对照，方便验证逻辑。可以改进的地方是目前把全部字符都压栈后再比较，若采用双指针法则不需要额外栈空间。

还有哪些实现方法？

直接从两端向中间扫描。

【问题描述】

题目要求检查源程序中的花括号 { 和 } 是否匹配，并分别用顺序栈和链栈实现。本实验在 Stacks/ex_5 中逐字符扫描文本，遇到左花括号就压栈，遇到右花括号就弹栈；若弹栈失败或扫描结束后栈不空，都说明括号不匹配。

【基本思路】

当扫描到某个右花括号时，如果当前栈已经为空，说明前面缺少左花括号；而文件读到末尾后如果栈还不为空，则说明还有未闭合的左花括号。这一逻辑正长对应了题目中的四条提示。

for ch in text: 若 ch=={ 则 push；若 ch==} 且栈空则失败，否则 pop；循环结束后若栈空则匹配成功。

函数调用关系可表示为：main() -> brackets_match_seq()/brackets_match_link() -> 输出每段文本的匹配结果。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

 本题代码位于 Stacks/ex_5/src/lib.rs，主程序选取了三段示例文本，分别对应完全匹配、缺少右括号和多余右括号三种情况。
 测试数据重点验证了空文本、嵌套花括号、只有右括号、缺少右括号等边界情形。

1.2 测试结果

运行 cargo test 后，本题 2 个单元测试全部通过；运行 cargo run 后，第一段代码的顺序栈和链栈检查结果都为 true，而另外两段错误文本都返回 false。

编号	测试数据	预期结果 / 实际分析
1	fn main() { let a = { 3 + 5 }; }	所有花括号配对成功，两种实现都返回 true。
2	if (a > b) { println!("ok");	末尾缺少右花括号，两种实现都返回 false。
3	}}	一开始就出现多余右花括号，立即判定不匹配。

2. 结果及方案分析

2.1 本实验的重点及难点

本题核心是“遇左压栈、遇右出栈”的对应关系。由于这里只检查花括号，所以实现较短，但也足够体现栈对成对符号匹配的作用。

2.2 本实验设计的优点及可以改进的方向

优点是逻辑清楚、判断条件少。可以改进的方向是把 ()、[] 等其它括号类型一起纳入检查。

队列基础练习一

【问题描述】

完成初始化、入队、出队、判空和判满等基本操作。本实验在 Queues/ex_1 中按数组循环使用的方式实现 CircularQueue，数组长度为 1024，因此最多可保存 1023 个整数。

【基本思路】

循环队列同时维护 front 和 rear 两个下标。front 指向队头元素，rear 指向下一个可插入位置。为了区分“空”和“满”，本实现保留一个空槽，所以当 $(rear + 1) \% MAX == front$ 时判定为满。

入队时先把元素写入 data[rear]，再把 rear 按模 MAX 前移；出队时先取出 data[front]，再把 front 前移。这和教材中的循环数组队列定义基本一致。

enqueue(x): 若队满则失败，否则 $data[rear] \leftarrow x$, $rear \leftarrow (rear+1) \bmod MAX$; dequeue() 类似地移动 front。

函数调用关系可表示为：main() -> CircularQueue::new() -> enqueue()/dequeue()/output()。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本题代码位于 Queues/ex_1/src/lib.rs，主程序演示了依次入队 11、22、33，再出队 1 次后的结果。

测试数据覆盖了空队、正常出入队、压入 1023 个元素后的判满，以及出队一项后继续入队的循环复用场景。

1.2 测试结果

运行 cargo test 后，本题 2 个单元测试全部通过；运行 cargo run 后，程序输出循环队列初始为空，入队后为 [11, 22, 33]，出队一个元素 11 后剩余 [22, 33]。

编号	测试数据	预期结果 / 实际分析
1	入队 11、22、33	队列内容应为 [11, 22, 33]。
2	出队 1 次	返回 11，队列变为 [22, 33]。
3	连续入队 1023 个元素后再入队	判满成功，额外入队被拒绝。

2. 结果及方案分析

2.1 本实验的重点及难点

front 和 rear 的移动规则，尤其要记住“队满时要空出一个位置”这个处理方法。若把队满条件写错，很容易和空队列混淆。

2.2 本实验设计的优点及可以改进的方向

优点是所有操作都能在 $O(1)$ 时间内完成，且空间连续。可以改进的地方是加入队列长度计算接口。

队列基础练习二

【问题描述】

题目要求根据结点结构构造链队列，并完成与表 10-9 相同的基本操作。本实验在 `Queues/ex_2` 中用单链表顺序连接所有队列元素，入队时把新结点挂到队尾，出队时从队头删除。

【基本思路】

链队列中每个结点保存一个整数 `data` 和后继 `next`。入队时从头走到尾后挂接新结点，出队时删除头结点。

和链栈类似，链队列理论上也不需要判断“队满”，因此 `is_full()` 固定返回 `false`。

`enqueue(x)` 时若队空则 `head <- 新结点`，否则遍历到尾部后 `tail.next <- 新结点`；`dequeue()` 时删除头结点。

函数调用关系可表示为：`main() -> LinkQueue::new() -> enqueue()/dequeue()/output()/len()`。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本题代码位于 `Queues/ex_2/src/lib.rs`，主程序演示了依次入队 7、14、21，再出队 1 次后的效果。

测试数据覆盖了正常先进先出、连续出队以及压入较多元素后 `is_full()` 依旧返回 `false` 的场景。

1.2 测试结果

运行 `cargo test` 后，本题 2 个单元测试全部通过；运行 `cargo run` 后，链队列内容为 `[7, 14, 21]`，出队元素为 7，剩余队列为 `[14, 21]`。

编号	测试数据	预期结果 / 实际分析
1	入队 7、14、21	输出 <code>[7, 14, 21]</code> ，顺序正确。
2	出队 1 次	返回 7，剩余 <code>[14, 21]</code> 。
3	连续入队 100 个元素	链队列可继续工作， <code>is_full()</code> 返回 <code>false</code> 。

2. 结果及方案分析

2.1 本实验的重点及难点

本题需要抓住队列“先进先出”的特征，因此出队一定发生在头部，入队一定发生在尾部。若把这两个方向写反，队列就会退化成栈。

2.2 本实验设计的优点及可以改进的方向

当前实现的优点是结构简单。缺点是没有记录尾指针，入队时需要遍历到链表末尾，效率不算高。

还有哪些实现方法？

同时维护 front 和 rear 两个指针，把入队时间降为 $O(1)$ 。

队列进阶练习一

【问题描述】

题目要求设计一个算法模拟病人排队就诊过程，满足“病人到达时入队，护士让下一位病人就诊时出队”的先进先服务原则。本实验在 Queues/ex_3 中实现了一个字符串链队列 PatientQueue，并提供 simulate_hospital() 解析 arrive、next、status 三类命令。

【基本思路】

PatientQueue 的结构和前一道链队列类似，只是结点里保存的是病人姓名字符串。
simulate_hospital() 顺序读取命令数组：遇到 arrive 张三 这样的命令就把病人名字入队；遇到 next 就让队头病人出队进入诊室；遇到 status 则输出当前等待人数和队列内容。

for cmd in commands: 若以 arrive 开头则入队；若 cmd 为 next 则队头出队；若为 status 则输出当前队列。

函数调用关系可表示为：main() -> simulate_hospital() -> PatientQueue::enqueue()/dequeue()/output()/len()。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本题代码位于 Queues/ex_3/src/lib.rs，主程序演示了三位病人依次到达、随后按先后顺序进入诊室的完整过程。
测试数据验证了 FIFO 顺序、队列状态输出，以及在无病人等待时执行 next 命令会返回“当前没有病人等待”。

1.2 测试结果

运行 cargo test 后，本题 2 个单元测试全部通过；运行 cargo run 后，程序依次输出张三、李四、王五到达，随后按照张三、李四、王五的顺序进入诊室，最后提示当前没有病人等待。

编号	测试数据	预期结果 / 实际分析
1	arrive 张三 -> arrive 李四 -> arrive 王五	等待队列依次变为 ["张三"]、["张三","李四"]、["张三","李四","王五"]。
2	next -> status	张三先进入诊室，随后剩余等待人数为 2，队列为 ["李四","王五"]。
3	连续执行 3 次 next 后再 next	李四、王五按顺序进入诊室，最后输出“当前没有病人等待”。

2. 结果及方案分析

2.1 本实验的重点及难点

arrive 命令中的姓名提取，以及 next 时空队列的处理。

2.2 本实验设计的优点及可以改进的方向

可以改进的地方是目前仍未保存尾指针，入队效率和前题一样是 $O(n)$ 。